Algoritmi numerici

Sapienza Università di Roma

Indice

Introduzione

Cambio di base

Test di primalità

Crivello di Eratostene

Massimo comun divisore

In sintesi

Introduzione

Quattro algoritmi classici sui numeri naturali

In questo capitolo studiamo quattro algoritmi classici che operano sui numeri naturali:

- il **cambio di base**, che scrive un numero naturale dato in base dieci come sequenza di cifre in un'altra base b;
- il **test di primalità**, che decide se un numero naturale è primo;
- il **crivello di Eratostene**, che trova tutti i numeri primi fino a un dato numero naturale;
- l'**algoritmo di Euclide**, che calcola il massimo comun divisore di due numeri naturali.

Una sola operazione elementare

- Tutti e quattro gli algoritmi si basano su una sola operazione elementare, la **divisione con resto**:
  - il cambio di base legge le cifre come resti;
  - il test di primalità e il crivello cercano resti uguali a zero;
  - l'algoritmo di Euclide sostituisce una coppia di numeri con una coppia più piccola per mezzo di un resto.

- Per ogni algoritmo presentiamo lo pseudocodice (con array indicizzati a partire da 1) e un'esecuzione passo per passo

- Sotto ogni pseudocodice, i link «Esegui in Colab» aprono la sua implementazione in Python, C++ e Java

Cambio di base

Introduzione: Cambio di base

- Sappiamo leggere una sequenza di cifre in base b come un numero: ora facciamo il **contrario**
- Dato un numero in base dieci, calcoliamo le sue cifre in una qualsiasi base $b \geq 2$ (per esempio, le cifre binarie usate da un calcolatore)
- Lo strumento è la divisione con resto: una divisione estrae una cifra
- Dimostriamo perché funziona, scriviamo l'algoritmo e lo eseguiamo su quattro numeri

Dalle cifre ai numeri, e ritorno

- Nel capitolo «Rappresentazione dei numeri» abbiamo visto che una sequenza di k cifre $(d_{k-1} \dots d_1 d_0)_b$ in base b rappresenta il numero naturale

$$\sum_{i=0}^{k-1} d_i \, b^i$$

- Abbiamo visto anche che servono $\lfloor \log_b n \rfloor + 1$ cifre per scrivere un numero naturale $n \geq 1$ in base b

- In questa sezione risolviamo il **problema inverso**: dato n in base dieci, calcoliamo le sue cifre in base b

Definizione 1: divisione con resto

Dati un numero naturale n e un numero naturale $b \geq 1$, la ***divisione con resto*** *(o divisione euclidea) di n per b determina i due numeri naturali q e r tali che*

$$n = q\ b + r \qquad \textit{con} \qquad 0 \leq r < b.$$

Il numero n si chiama ***dividendo***, *b* ***divisore***, *q* ***quoziente*** *e r* ***resto***. *I due numeri q e r sono unici, e scriviamo*

$$q = \lfloor n/b \rfloor \qquad e \qquad r = n \bmod b.$$

Il fatto chiave

Una divisione per b estrae
la **cifra meno significativa** di n in base b,

e lascia come quoziente il numero
formato dalle **cifre rimanenti**.

Osservazione 1

Sia $n \geq 1$ un numero naturale la cui rappresentazione in base $b \geq 2$ è $(d_{k-1} \dots d_1 d_0)_b$. Allora

$$n \bmod b = d_0 \qquad e$$

$$\lfloor n/b \rfloor = \sum_{i=1}^{k-1} d_i \, b^{i-1} = (d_{k-1} \dots d_1)_b,$$

dove il quoziente è 0 se $k = 1$.

Dimostrazione

Raccogliendo il fattore b da tutti i termini con $i \geq 1$, otteniamo

$$n = \sum_{i=0}^{k-1} d_i \, b^i = \left(\sum_{i=1}^{k-1} d_i \, b^{i-1} \right) b + d_0.$$

- Poiché d_0 è una cifra in base b, si ha $0 \leq d_0 < b$
- Per l'unicità del quoziente e del resto della divisione con resto, il quoziente di n diviso b è $\sum_{i=1}^{k-1} d_i \, b^{i-1}$ e il resto è d_0

□

Ripetere la divisione

- Dividendo n per b, il resto è la cifra meno significativa d_0, e il quoziente è il numero scritto con le altre cifre $(d_{k-1} \dots d_1)_b$
- Dividendo di nuovo il quoziente per b si ottiene d_1, e così via
- Ripetendo la divisione fino a ottenere quoziente 0 si producono tutte le cifre di n in base b, dalla meno significativa alla più significativa, in esattamente $\lfloor \log_b n \rfloor + 1$ divisioni

Il problema

Dato un numero naturale $n > 0$ rappresentato in base 10 (decimale)
e una base di arrivo $b \geq 2$,
consideriamo il problema di convertire n in base b.

- Segue lo pseudocodice dell'algoritmo per convertire sequenze di cifre dalla base dieci alla base b

Algorithm: ChangeOfBase (1/2)

Input : Un numero naturale $n \geq 1$ (in base 10) e una base $b \geq 2$

Output : Un array $d[1..nd]$ di cifre in base b e il numero di cifre nd

```
1 function ChangeOfBase(n, b)
2 |   nd := ⌊log_b n⌋ + 1    // Numero di cifre necessarie
3 |   d := new array[1..nd]
4 |   k := 1
```

Algorithm: ChangeOfBase (2/2)

```
function ChangeOfBase(n, b) (continua)
5     while n > 0 do
6         q := ⌊n/b⌋                                  // Quoziente
7         r := n mod b                  // Resto (cifra in base b)
8         d[k] ← r
9         n ← q
10        k ← k + 1
11    end
12    return d, nd
13 end
```

Esegui in Colab: Python · C++ · Java

Come funziona ChangeOfBase (1/2)

- L'algoritmo calcola il numero di cifre nd necessarie per rappresentare n in base b con la formula $\lfloor \log_b n \rfloor + 1$
- Inizializza un array $d[1..nd]$ per memorizzare le cifre e un contatore k che tiene traccia della posizione corrente
- Il ciclo principale divide ripetutamente n per b per estrarre le cifre da destra a sinistra (dalla meno significativa alla più significativa)
- A ogni iterazione il resto $r = n \bmod b$ fornisce la cifra successiva, che viene memorizzata in $d[k]$

Come funziona ChangeOfBase (2/2)

- Il quoziente $q = \lfloor n/b \rfloor$ diventa il nuovo valore di n per l'iterazione successiva
- L'algoritmo termina quando $n = 0$, restituendo l'array d e il numero totale di cifre nd
- Le cifre in d sono memorizzate in **ordine inverso**: l'elemento $d[k]$ è la cifra d_{k-1} di peso b^{k-1}, quindi
  - $d[1] = d_0$ è la cifra meno significativa (la più a destra)
  - $d[nd] = d_{nd-1}$ è la cifra più significativa (la più a sinistra)

Eseguiamo ora l'algoritmo su quattro numeri:
7 e 8 in base 2, 25 in base 3, 100 in base 8.

Esempio (1/4)

Conversione di 7 in base $b = 2$:

iterazione k	n	n/b	q	r	cifra
1	7	$7/2$	3	1	$d[1] = d_0$ (meno significativa)
2	3	$3/2$	1	1	$d[2] = d_1$
3	1	$1/2$	0	1	$d[3] = d_2$ (più significativa)

Quindi 7 in base 2 è $(111)_2$, con $nd = \lfloor \log_2 7 \rfloor + 1 = 3$ cifre.

Esempio (2/4)

Conversione di 8 in base $b = 2$:

iterazione k	n	n/b	q	r	cifra
1	8	$8/2$	4	0	$d[1] = d_0$ (meno significativa)
2	4	$4/2$	2	0	$d[2] = d_1$
3	2	$2/2$	1	0	$d[3] = d_2$
4	1	$1/2$	0	1	$d[4] = d_3$ (più significativa)

Quindi 8 in base 2 è $(1000)_2$, con $nd = \lfloor \log_2 8 \rfloor + 1 = 4$ cifre.

Esempio (3/4)

Conversione di 25 in base $b = 3$:

iterazione k	n	n/b	q	r	cifra
1	25	$25/3$	8	1	$d[1] = d_0$ (meno significativa)
2	8	$8/3$	2	2	$d[2] = d_1$
3	2	$2/3$	0	2	$d[3] = d_2$ (più significativa)

Quindi 25 in base 3 è $(221)_3$, con $nd = \lfloor \log_3 25 \rfloor + 1 = 3$ cifre.

Esempio (4/4)

Conversione di 100 in base $b = 8$:

iterazione k	n	n/b	q	r	cifra
1	100	$100/8$	12	4	$d[1] = d_0$ (meno significativa)
2	12	$12/8$	1	4	$d[2] = d_1$
3	1	$1/8$	0	1	$d[3] = d_2$ (più significativa)

Quindi 100 in base 8 è $(144)_8$, con $nd = \lfloor \log_8 100 \rfloor + 1 = 3$ cifre.

Riepilogo

Cambio di base:

- Il resto della divisione di n per b è la cifra meno significativa di n in base b; il quoziente è il numero scritto con le altre cifre
- L'algoritmo esegue una divisione per ogni cifra: $\lfloor \log_b n \rfloor + 1$ iterazioni
- Le cifre vengono prodotte dalla meno significativa alla più significativa, quindi il risultato si legge al contrario: da $d[nd]$ a $d[1]$

Test di primalità

Introduzione: Test di primalità

- I numeri primi sono i mattoni dei numeri naturali, e decidere se un numero è primo è un compito di base (per esempio, in crittografia)
- Partiamo da un algoritmo diretto suggerito dalla definizione
- Poi dimostriamo che basta provare i candidati divisori fino a $\sqrt{n}$: molte meno divisioni

Quando il resto è zero

- Il cambio di base usa il resto n mod b per leggere una cifra
- Un resto uguale a zero ha un significato proprio: dice che la divisione è esatta, cioè che b è un **divisore** di n
- Contare i divisori porta ai **numeri primi**

Definizione 2: divisore

Un numero naturale $d \geq 1$ è un ***divisore*** *di un numero naturale n se esiste un numero naturale k tale che $n = k\ d$:*

$$d \text{ è un divisore di } n \iff \exists\, k \in \mathbb{N},\ n = k\ d$$

o, equivalentemente, se $n \bmod d = 0$.

Definizione 3: numero primo

*Un numero naturale $n \geq 2$ è **primo** se i suoi unici divisori sono 1 e n stesso. Un numero naturale $n \geq 2$ che non è primo si dice **composto**: ha un divisore d con $2 \leq d \leq n-1$. I numeri 0 e 1 non sono primi.*

Il problema

Dato un numero naturale n,
consideriamo il problema di stabilire
se n è un numero primo.

Un primo algoritmo

- La definizione suggerisce un algoritmo diretto: provare, uno per uno, i candidati divisori di n diversi da 1 e da n
- L'osservazione seguente mostra che i candidati maggiori di $n/2$ si possono saltare

Osservazione 2

Per ogni numero naturale $n \geq 2$, se d è un divisore di n con $d < n$, allora $d \leq n/2$.

Dimostrazione

- Poiché d è un divisore di n, esiste un numero naturale k tale che $n = k\ d$
- Poiché $d < n$, si ha $k \geq 2$, quindi

$$d = \frac{n}{k} \leq \frac{n}{2}.$$

□

- Ne segue che basta controllare se qualche numero naturale d in $\{2, 3, \ldots, \lfloor n/2 \rfloor\}$ è un divisore di n
- Se non esiste alcun divisore di questo tipo, n è primo

Algorithm: BasicPrimalityTest

Input : Un numero naturale n

Output : `true` se n è primo, `false` altrimenti

```
1  function BasicPrimalityTest(n)
2      if n < 2 then
3          return false              // 0 e 1 non sono primi
4      end
5      for d := 2 to ⌊n/2⌋ do
6          if n mod d = 0 then
7              return false          // d divide n: non primo
8          end
9      end
10     return true                   // Nessun divisore trovato: n è
        primo
11 end
```

Esegui in Colab: Python · C++ · Java

Come funziona BasicPrimalityTest

- I numeri minori di 2 non sono primi per definizione
- Il ciclo prova ogni candidato divisore d da 2 fino a $\lfloor n/2 \rfloor$
- Se d è un divisore di n (cioè $n \bmod d = 0$), la funzione restituisce subito `false`
- Se il ciclo termina senza trovare alcun divisore, n è primo e la funzione restituisce `true`

- Per un numero primo il ciclo viene eseguito per intero: per $n = 29$ l'algoritmo prova i 13 candidati $d = 2, 3, \ldots, 14$
- Mostriamo ora che bastano molti meno candidati

Provare i divisori fino a $\sqrt{n}$

I divisori di un numero naturale si presentano a coppie:
d e n/d.

In ogni coppia, uno dei due numeri è al più $\sqrt{n}$.

Osservazione 3

Per ogni numero naturale $n \geq 2$, se d è un divisore di n allora n/d è un divisore di n.

Dimostrazione

Se d è un divisore di n, allora esiste un numero naturale k tale che

$$n = k\ d.$$

Poiché $k = n/d$, ne segue che n/d è un numero naturale. Quindi

$$n = d\left(\frac{n}{d}\right),$$

quindi anche n/d è un divisore di n. □

Osservazione 4

Per ogni numero naturale $n \geq 2$, se $d > \sqrt{n}$ allora $n/d < \sqrt{n}$.

Dimostrazione

Se $d > \sqrt{n}$, dividendo n per entrambi i membri di questa disuguaglianza otteniamo

$$\frac{n}{d} < \frac{n}{\sqrt{n}} = \sqrt{n}.$$

Quindi n/d è minore di $\sqrt{n}$. □

Corollario 1

Ogni numero naturale composto $n \geq 2$ ha un divisore d con $2 \leq d \leq \lfloor \sqrt{n} \rfloor$. Equivalentemente, un numero naturale $n \geq 2$ è primo se e solo se nessun numero naturale in $\{2, 3, \ldots, \lfloor \sqrt{n} \rfloor\}$ è un divisore di n.

Dimostrazione

- Poiché n è composto, ha un divisore d con $2 \leq d \leq n-1$
- Se $d \leq \sqrt{n}$, allora $d \leq \lfloor \sqrt{n} \rfloor$, perché d è un numero naturale, e abbiamo finito
- Altrimenti $d > \sqrt{n}$: per le due osservazioni precedenti, n/d è un divisore di n con $n/d < \sqrt{n}$
- Inoltre $n/d \geq 2$ perché $d < n$
- Quindi n/d è un divisore di n con $2 \leq n/d \leq \lfloor \sqrt{n} \rfloor$

□

I divisori si presentano a coppie

- Dalle osservazioni precedenti sappiamo che, per ogni numero naturale $n \geq 2$, se $d > \sqrt{n}$ è un divisore di n, allora anche n/d è un divisore di n e $n/d < \sqrt{n}$
- Ne segue che ogni divisore di n maggiore di $\sqrt{n}$ è accoppiato a un divisore di n minore di $\sqrt{n}$
- Quindi, se n ha un qualche divisore (diverso da 1 e da n), almeno uno di essi deve essere $\leq \sqrt{n}$

Esempio

Consideriamo $n = 36$: $\sqrt{36} = 6$. I divisori di 36 diversi da 1 e da 36 sono $2, 3, 4, 6, 9, 12, 18$.

divisore $d > \sqrt{36}$	divisore accoppiato $36/d < \sqrt{36}$
9	$36/9 = 4$
12	$36/12 = 3$
18	$36/18 = 2$

Ogni divisore > 6 è accoppiato a un divisore < 6; il divisore $6 = \sqrt{36}$ è accoppiato con se stesso.

Un algoritmo più veloce

- Per il corollario, il ciclo dell'algoritmo di base può fermarsi a $\lfloor \sqrt{n} \rfloor$ invece che a $\lfloor n/2 \rfloor$
- Segue lo pseudocodice dell'algoritmo Primality Test (test di primalità)

Algorithm: PrimalityTest

Input : Un numero naturale n
Output : `true` se n è primo, `false` altrimenti

```
1  function PrimalityTest(n)
2      if n < 2 then
3          return false            // 0 e 1 non sono primi
4      end
5      d := 2
6      while d ≤ ⌊√n⌋ do
7          if n mod d = 0 then
8              return false        // d divide n:  non primo
9          end
10         d ← d + 1
11     end
12     return true      // Nessun divisore trovato:  n è
        primo
13 end
```

Esegui in Colab: Python · C++ · Java

Come funziona PrimalityTest

- I numeri minori di 2 non sono primi per definizione
- Il ciclo prova ogni candidato divisore d da 2 fino a $\lfloor\sqrt{n}\rfloor$
- Se d divide n (cioè $n \bmod d = 0$), la funzione restituisce subito `false`
- Se il ciclo termina senza trovare alcun divisore, allora, per il corollario, n è primo e la funzione restituisce `true`
- Per $n = 2$ e $n = 3$ si ha $\lfloor\sqrt{n}\rfloor = 1 < 2$: il ciclo non viene eseguito e la funzione restituisce `true`

Esempio (1/3)

Verifica $n = 36$: $\lfloor \sqrt{36} \rfloor = 6$

d	36 mod d	azione
2	0	restituisce `false`
36 non è primo $(36 = 2 \cdot 18)$.		

Esempio (2/3)

Verifica $n = 29$: $\lfloor \sqrt{29} \rfloor = 5$

d	$29 \bmod d$	azione
2	$1 \neq 0$	continua
3	$2 \neq 0$	continua
4	$1 \neq 0$	continua
5	$4 \neq 0$	continua
Il ciclo termina. Restituisce `true`: **29 è primo**.		

Esempio (3/3)

Verifica $n = 25$: $\lfloor \sqrt{25} \rfloor = 5$

d	$25 \bmod d$	azione
2	$1 \neq 0$	continua
3	$1 \neq 0$	continua
4	$1 \neq 0$	continua
5	0	restituisce `false`
25 non è primo $(25 = 5 \cdot 5)$.		

Per il numero primo 29 l'algoritmo prova 4 candidati, mentre l'algoritmo di base ne prova 13.

Riepilogo

Test di primalità:

- I divisori di n si presentano a coppie d e n/d, e in ogni coppia uno dei due numeri è al più $\sqrt{n}$
- Quindi $n \geq 2$ è primo se e solo se nessun numero naturale in $\{2, 3, \ldots, \lfloor\sqrt{n}\rfloor\}$ lo divide
- Il Primality Test esegue al più $\lfloor\sqrt{n}\rfloor - 1$ iterazioni, invece delle $\lfloor n/2\rfloor - 1$ iterazioni dell'algoritmo di base

Crivello di Eratostene

Introduzione: Crivello di Eratostene

- Ora vogliamo **tutti** i numeri primi fino a n, non solo la risposta per un numero
- Il crivello è uno degli algoritmi più antichi che si conoscano, e un modo semplice per costruire la tabella di tutti i primi fino a n
- Dimostriamo due proprietà che dicono dove la marcatura comincia e dove si ferma, poi eseguiamo l'algoritmo per $n = 10$ e $n = 20$

Cancellare i multipli

- Il Primality Test risponde alla domanda per un solo numero
- Quando servono tutti i numeri primi fino a n, chiamarlo su $2, 3, \ldots, n$ ripete molte divisioni
- Il crivello lavora al contrario: invece di cercare i divisori di ciascun numero, **cancella i multipli** di ciascun primo

Il problema

Dato un numero naturale n,
consideriamo il problema di determinare
tutti i numeri primi minori o uguali a n.

- Il **crivello di Eratostene** è un algoritmo antico (attribuito al matematico greco Eratostene di Cirene, ca. 276–194 a.C.) che calcola tutti i numeri primi fino a un numero naturale n
- L'idea chiave è partire con tutti i numeri naturali da 2 a n marcati come potenzialmente primi, e poi marcare iterativamente come composti i multipli di ciascun primo

Definizione 4: multiplo

Un numero naturale m è un ***multiplo*** *di un numero naturale $p \geq 1$ se esiste un numero naturale k tale che $m = k\ p$:*

$$m \text{ è un multiplo di } p \iff \exists\, k \in \mathbb{N},\ m = k\ p$$

o, equivalentemente, se $m \bmod p = 0$.

Dove comincia la marcatura

La prima osservazione dice da dove può partire la marcatura dei multipli di p.

Osservazione 5

Per ogni numero naturale $p \geq 2$, se un numero naturale m è un multiplo di p con $p < m < p^2$, allora m ha un divisore d con $2 \leq d < p$.

Dimostrazione

- Poiché m è un multiplo di p, esiste un numero naturale d tale che $m = d\ p$. Allora d è un divisore di m
- Inoltre, poiché $m > p$ e $m = d\ p$, si ha $d \geq 2$
- Poiché $m < p^2$ e $m = d\ p$, si ha $d < p$
- Quindi $2 \leq d < p$

□

Partire da p^2

- Dall'osservazione precedente, per ogni primo p e ogni multiplo m di p con $p < m < p^2$, il numero m ha un divisore d con $2 \leq d < p$
- Quindi m ha anche un divisore primo q con $2 \leq q < p$, ed era perciò già stato marcato come composto quando è stato elaborato q
- Dunque, nel marcare i multipli di p, si può partire da p^2

Esempio

Sia $p = 5$. I suoi multipli compresi tra p e $p^2 = 25$ sono $m = 10, 15, 20$.

multiplo m	$m = d \cdot p$	divisore $d < p$	già marcato quando
10	$10 = 2 \cdot 5$	$d = 2$	$p = 2$ (da $j = 4$, passo 2)
15	$15 = 3 \cdot 5$	$d = 3$	$p = 3$ (da $j = 9$, passo 3)
20	$20 = 4 \cdot 5$	$d = 4$	$p = 2$ (da $j = 4$, passo 2)

- Il numero 20 ha il divisore $d = 4$, che non è primo, e anche il divisore primo 2, che lo ha marcato
- Il multiplo successivo è $25 = 5 \cdot 5 = p^2$, che non è stato marcato da nessun primo precedente
- Quindi, per $p = 5$, si può iniziare a marcare i numeri da 25

Dove si ferma il ciclo esterno

La seconda osservazione dice dove può fermarsi il ciclo esterno su p.

Osservazione 6

Per ogni numero naturale $n \geq 2$, ogni numero naturale composto $m \leq n$ ha un divisore primo q con $q \leq \lfloor \sqrt{n} \rfloor$.

Dimostrazione

- Per il corollario della sezione «Test di primalità», m ha un divisore d con
$$2 \leq d \leq \lfloor \sqrt{m} \rfloor \leq \lfloor \sqrt{n} \rfloor$$
- Sia q il più piccolo divisore di m con $q \geq 2$; allora $q \leq d \leq \lfloor \sqrt{n} \rfloor$
- Il numero q è primo: se q avesse un divisore c con $2 \leq c < q$, allora c sarebbe anche un divisore di m (da $q = k_1 c$ e $m = k_2 q$ si ottiene $m = k_2 k_1 c$), contro la scelta di q come il più piccolo

□

Fermarsi a $\lfloor \sqrt{n} \rfloor$

- Quindi, dopo che sono stati marcati i multipli di tutti i primi $p \leq \lfloor \sqrt{n} \rfloor$, ogni numero composto fino a n è marcato, e il ciclo esterno può fermarsi
- Segue lo pseudocodice del crivello di Eratostene

Algorithm: SieveOfEratosthenes (1/2)

Input : Un numero naturale $n \geq 1$
Output : Un array booleano $prime[1..n]$ dove $prime[i] = 1$ se e solo se i è primo

```
1 function SieveOfEratosthenes(n)
2 |   prime := new array[1..n]          // dichiara l'array
  |    booleano
3 |   prime ← 1 // inizializza tutti gli elementi a 1
4 |   prime[1] ← 0                      // 1 non è primo
5 |   p := 2
```

Algorithm: SieveOfEratosthenes (2/2)

function *SieveOfEratosthenes(n)* (continua)

```
6     while p ≤ ⌊√n⌋ do
7         if prime[p] = 1 then
8             j := p²
9             while j ≤ n do
10                prime[j] ← 0                    // j non è primo
11                j ← j + p
12            end
13        end
14        p ← p + 1
15    end
16    return prime
17 end
```

Esegui in Colab: Python · C++ · Java

Come funziona SieveOfEratosthenes (1/2)

- L'array *prime* viene dichiarato e tutti i suoi elementi sono inizializzati a 1; l'elemento *prime*[1] viene subito posto a 0, poiché 1 non è primo
- Il ciclo esterno scorre ogni candidato primo p da 2 fino a $\lfloor \sqrt{n} \rfloor$
- Se $prime[p] = 1$, allora p è primo (un p composto sarebbe stato marcato dal suo più piccolo divisore primo), e tutti i suoi multipli a partire da p^2 sono posti a 0

Come funziona SieveOfEratosthenes (2/2)

- Il ciclo interno parte da p^2 perché tutti i multipli più piccoli $2p, 3p, \ldots, (p-1)p$ sono già stati posti a 0 nell'elaborare i primi più piccoli $2, 3, 5, 7, \ldots$
- Dopo l'esecuzione dell'algoritmo, $prime[i] = 1$ se e solo se i è primo

Esempio (1/2)

- Eseguiamo il crivello con $n = 10$, quindi $\lfloor \sqrt{10} \rfloor = 3$
- Ogni riga della tabella mostra lo stato dell'array $prime[1..10]$ dopo l'assegnamento $prime[j] \leftarrow 0$
- Le **celle blu** indicano gli elementi appena posti a 0
- Le **celle rosse** indicano il valore corrente di p

Esempio (2/2)

		prime[1..10]										
p	j	1	2	3	4	5	6	7	8	9	10	azione
–	–	0	1	1	1	1	1	1	1	1	1	inizializzazione
2	4	0	1	1	0	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	0	1	1	0	1	0	1	1	1	1	$prime[6] \leftarrow 0$
	8	0	1	1	0	1	0	1	0	1	1	$prime[8] \leftarrow 0$
	10	0	1	1	0	1	0	1	0	1	0	$prime[10] \leftarrow 0$
	12											$j = 12 > 10$: arresto
3	9	0	1	1	0	1	0	1	0	0	0	$prime[9] \leftarrow 0$
	12											$j = 12 > 10$: arresto
Il ciclo esterno termina: $p = 4 > \lfloor\sqrt{10}\rfloor = 3$.												

Risultato: i numeri primi ≤ 10 sono $2,\ 3,\ 5,\ 7$.

Esempio (1/5)

- Eseguiamo il crivello con $n = 20$, quindi $\lfloor \sqrt{20} \rfloor = 4$
- Ogni riga della tabella mostra lo stato dell'array $prime[1..20]$ dopo l'assegnamento $prime[j] \leftarrow 0$
- **Celle blu**: elemento appena posto a 0; **celle rosse**: valore corrente di p
- Quando l'elemento è già 0, l'assegnamento non modifica l'array
- L'array è troppo lungo per una slide: mostriamo gli elementi 1..10 e poi gli elementi 11..20

Esempio (2/5)

Il primo $p = 2$, elementi $prime[1..10]$:

		$prime[1..10]$										
p	j	1	2	3	4	5	6	7	8	9	10	azione
–	–	0	1	1	1	1	1	1	1	1	1	inizializzazione
2	4	0	1	1	0	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	0	1	1	0	1	0	1	1	1	1	$prime[6] \leftarrow 0$
	8	0	1	1	0	1	0	1	0	1	1	$prime[8] \leftarrow 0$
	10	0	1	1	0	1	0	1	0	1	0	$prime[10] \leftarrow 0$
	12	0	1	1	0	1	0	1	0	1	0	$prime[12] \leftarrow 0$
	14	0	1	1	0	1	0	1	0	1	0	$prime[14] \leftarrow 0$
	16	0	1	1	0	1	0	1	0	1	0	$prime[16] \leftarrow 0$
	18	0	1	1	0	1	0	1	0	1	0	$prime[18] \leftarrow 0$
	20	0	1	1	0	1	0	1	0	1	0	$prime[20] \leftarrow 0$
	22											$j = 22 > 20$: arresto

Esempio (3/5)

Il primo $p = 2$, elementi $prime[11..20]$:

p	j	$prime[11..20]$										azione
		11	12	13	14	15	16	17	18	19	20	
–	–	1	1	1	1	1	1	1	1	1	1	inizializzazione
2	4	1	1	1	1	1	1	1	1	1	1	$prime[4] \leftarrow 0$
	6	1	1	1	1	1	1	1	1	1	1	$prime[6] \leftarrow 0$
	8	1	1	1	1	1	1	1	1	1	1	$prime[8] \leftarrow 0$
	10	1	1	1	1	1	1	1	1	1	1	$prime[10] \leftarrow 0$
	12	1	0	1	1	1	1	1	1	1	1	$prime[12] \leftarrow 0$
	14	1	0	1	0	1	1	1	1	1	1	$prime[14] \leftarrow 0$
	16	1	0	1	0	1	0	1	1	1	1	$prime[16] \leftarrow 0$
	18	1	0	1	0	1	0	1	0	1	1	$prime[18] \leftarrow 0$
	20	1	0	1	0	1	0	1	0	1	0	$prime[20] \leftarrow 0$
	22											$j = 22 > 20$: arresto

Esempio (4/5)

I valori $p = 3$ e $p = 4$, elementi $prime[1..10]$:

p	j	$prime[1..10]$ 1	2	3	4	5	6	7	8	9	10	azione
3	9	0	1	1	0	1	0	1	0	0	0	$prime[9] \leftarrow 0$
	12	0	1	1	0	1	0	1	0	0	0	$prime[12] \leftarrow 0$ (già 0)
	15	0	1	1	0	1	0	1	0	0	0	$prime[15] \leftarrow 0$
	18	0	1	1	0	1	0	1	0	0	0	$prime[18] \leftarrow 0$ (già 0)
	21											$j = 21 > 20$: arresto
4												$prime[4] = 0$: salta

Il ciclo esterno termina: $p = 5 > \lfloor\sqrt{20}\rfloor = 4$.

Esempio (5/5)

I valori $p = 3$ e $p = 4$, elementi $prime[11..20]$:

p	j	$prime[11..20]$ 11	12	13	14	15	16	17	18	19	20	azione
3	9	1	0	1	0	1	0	1	0	1	0	$prime[9] \leftarrow 0$
	12	1	0	1	0	1	0	1	0	1	0	$prime[12] \leftarrow 0$ (già 0)
	15	1	0	1	0	0	0	1	0	1	0	$prime[15] \leftarrow 0$
	18	1	0	1	0	0	0	1	0	1	0	$prime[18] \leftarrow 0$ (già 0)
	21											$j = 21 > 20$: arresto
4												$prime[4] = 0$: salta

Il ciclo esterno termina: $p = 5 > \lfloor \sqrt{20} \rfloor = 4$.

Risultato: i numeri primi ≤ 20 sono 2, 3, 5, 7, 11, 13, 17, 19.

Riepilogo

Crivello di Eratostene:

- Il crivello non divide: marca come composti i multipli di ciascun primo p
- La marcatura dei multipli di p parte da p^2: i multipli più piccoli hanno un divisore primo più piccolo e sono già marcati
- Il ciclo esterno si ferma a $\lfloor \sqrt{n} \rfloor$: ogni numero composto fino a n ha un divisore primo non maggiore di $\lfloor \sqrt{n} \rfloor$

Massimo comun divisore

Introduzione: Massimo comun divisore

- L'ultimo algoritmo del capitolo tratta **due numeri alla volta**: cerchiamo i divisori che hanno in comune
- Il massimo comun divisore serve, per esempio, a ridurre una frazione ai minimi termini
- Dimostriamo una proprietà che sostituisce una coppia di numeri con una coppia più piccola, e da essa segue l'**algoritmo di Euclide**
- Alla fine lo usiamo per decidere se due numeri sono coprimi

Definizione 5: divisore comune e massimo comun divisore (1/2)

Siano n, m due numeri naturali con $n, m \geq 1$.

- *Un numero naturale $d \geq 1$ è un **divisore comune** di n e m se d è un divisore sia di n sia di m, cioè se esistono numeri naturali k_1, k_2 tali che*

$$n = k_1\ d \qquad e \qquad m = k_2\ d.$$

Equivalentemente, $n \bmod d = 0$ e $m \bmod d = 0$.

Definizione: divisore comune e massimo comun divisore (2/2)

- *Il* ***massimo comun divisore*** *di* n *e* m*, indicato con* $\gcd(n, m)$*, è il più grande divisore comune di* n *e* m*.*

Esistenza del massimo comun divisore

- Il massimo comun divisore esiste sempre: 1 è un divisore comune, e ogni divisore comune è al più $\min\{n, m\}$
- Per esempio, i divisori comuni di 48 e 18 sono $1, 2, 3, 6$, quindi $\gcd(48, 18) = 6$

Dati due numeri naturali n, m con $n, m \geq 1$,
consideriamo il problema di calcolare
il loro massimo comun divisore.

Una coppia più piccola

L'osservazione seguente sostituisce la coppia (n, m) con una coppia di numeri più piccoli che ha lo stesso massimo comun divisore.

Osservazione 7

Per ogni coppia di numeri naturali $n \geq 1$ e $m \geq 1$,

$$\gcd(n, m) = \gcd(m,\ n \bmod m).$$

Dimostrazione (1/3)

Sia $q = \lfloor n/m \rfloor$, in modo che $n = qm + (n \bmod m)$.

Sia d_1 un divisore comune di n e m. Allora esistono numeri naturali k_1, k_2 tali che

$$n = k_1 d_1 \qquad \text{e} \qquad m = k_2 d_1.$$

Poiché $n \bmod m = n - qm$, otteniamo

$$n \bmod m = (k_1 - qk_2)\, d_1,$$

dove $k_1 - qk_2 \geq 0$ perché $n \bmod m \geq 0$. Quindi d_1 è anche un divisore di $n \bmod m$. Perciò d_1 è un divisore comune di m e $n \bmod m$.

Dimostrazione (2/3)

Viceversa, sia d_2 un divisore comune di m e $n \bmod m$. Allora esistono numeri naturali k_1, k_2 tali che

$$m = k_1 d_2 \qquad \text{e} \qquad n \bmod m = k_2 d_2.$$

Poiché $n = qm + (n \bmod m)$, otteniamo

$$n = (qk_1 + k_2)\, d_2,$$

quindi d_2 è anche un divisore di n. Perciò d_2 è un divisore comune di n e m.

Dimostrazione (3/3)

Dunque n e m hanno esattamente gli stessi divisori comuni di m e $n \bmod m$, e quindi

$$\gcd(n, m) = \gcd(m,\ n \bmod m).$$

Nel membro di destra, $\gcd(m, 0)$ è inteso come il più grande divisore di m (ogni $d \geq 1$ divide $0 = 0 \cdot d$), cioè m stesso. □

Algoritmo di Euclide

- L'**algoritmo di Euclide** (attribuito all'antico matematico greco Euclide, ca. 300 a.C.) calcola $\gcd(n, m)$ in modo efficiente sostituendo ripetutamente la coppia (n, m) con $(m,\ n \bmod m)$

- L'algoritmo non richiede $n \geq m$: se $n < m$, allora $n \bmod m = n$, e la prima sostituzione scambia semplicemente i due numeri, trasformando (n, m) in (m, n)

Perché la sostituzione funziona

- Per l'osservazione precedente, sostituire (n, m) con $(m,\ n \bmod m)$ non cambia il massimo comun divisore
- Poiché $n \bmod m < m$, il secondo argomento decresce strettamente a ogni sostituzione
- Dunque l'algoritmo termina quando il resto diventa 0; a quel punto il massimo comun divisore è l'altro argomento:

$$\gcd(n, 0) = n.$$

Algorithm: EuclideanAlgorithm

Input : Due numeri naturali $n, m \geq 1$
Output: $\gcd(n, m)$

```
1 function EuclideanAlgorithm(n, m)
2     while m ≠ 0 do
3         r := n mod m
4         n ← m
5         m ← r
6     end
7     return n                    // m = 0:  n è il MCD
8 end
```

Esegui in Colab: Python · C++ · Java

Come funziona EuclideanAlgorithm

- A ogni passo, (n, m) viene sostituita da $(m,\ n \bmod m)$
- Per l'osservazione precedente, questa sostituzione conserva il valore di $\gcd(n, m)$: indicando con n_0 e m_0 i valori in input, a ogni iterazione $\gcd(n, m) = \gcd(n_0, m_0)$
- Poiché $n \bmod m < m$, il valore di m decresce strettamente a ogni iterazione, quindi l'algoritmo termina
- Quando $m = 0$, l'uguaglianza $\gcd(n, m) = \gcd(n_0, m_0)$ insieme a $\gcd(n, 0) = n$ dà il risultato

Esempio (1/2)

Calcoliamo $\gcd(48, 18)$:

passo	n	m	$r = n \bmod m$
1	48	18	$48 \bmod 18 = 12$
2	18	12	$18 \bmod 12 = 6$
3	12	6	$12 \bmod 6 = 0$
4	6	0	— (il ciclo termina)

Risultato: $\gcd(48, 18) = 6$.

Esempio (2/2)

Calcoliamo $\gcd(252, 105)$:

passo	n	m	$r = n \bmod m$
1	252	105	$252 \bmod 105 = 42$
2	105	42	$105 \bmod 42 = 21$
3	42	21	$42 \bmod 21 = 0$
4	21	0	— (il ciclo termina)

Risultato: $\gcd(252, 105) = 21$.

Coprimalità

Definizione 6: numeri coprimi

*Due numeri naturali $n, m \geq 1$ si dicono **coprimi** (o **primi tra loro**) se non hanno alcun divisore comune diverso da 1:*

$$n \text{ e } m \text{ sono coprimi} \iff \gcd(n, m) = 1.$$

- Poiché l'algoritmo di Euclide calcola $\gcd(n, m)$, per verificare la coprimalità basta chiamare l'algoritmo e controllare se il risultato è uguale a 1

Esempio

- Calcoliamo $\gcd(35, 12)$: l'algoritmo di Euclide restituisce 1. Quindi 35 e 12 sono **coprimi**
- Calcoliamo $\gcd(48, 18)$: l'algoritmo di Euclide restituisce 6. Quindi 48 e 18 **non sono coprimi**

Riepilogo

Massimo comun divisore:

- Le coppie (n, m) e $(m,\ n \bmod m)$ hanno gli stessi divisori comuni, quindi lo stesso massimo comun divisore
- L'algoritmo di Euclide ripete la sostituzione finché il secondo numero è 0, e restituisce il primo, poiché $\gcd(n, 0) = n$
- Due numeri sono coprimi se e solo se l'algoritmo di Euclide restituisce 1

In sintesi

Riepilogo (1/2)

Divisione con resto: $n = q\,b + r$ con $0 \leq r < b$; tutti e quattro gli algoritmi si basano su di essa

Cambio di base:

- ogni divisione per b dà come resto la cifra successiva, a partire dalla meno significativa
- $\lfloor \log_b n \rfloor + 1$ divisioni; le cifre si leggono al contrario

Test di primalità:

- i divisori si presentano a coppie d e n/d: basta provare fino a $\lfloor \sqrt{n} \rfloor$
- al più $\lfloor \sqrt{n} \rfloor - 1$ iterazioni invece di $\lfloor n/2 \rfloor - 1$

Riepilogo (2/2)

Crivello di Eratostene:

- marca i multipli di ciascun primo p, a partire da p^2
- il ciclo esterno si ferma a $\lfloor \sqrt{n} \rfloor$

Algoritmo di Euclide:

- $\gcd(n, m) = \gcd(m,\ n \bmod m)$, finché il secondo numero è 0; poi $\gcd(n, 0) = n$
- n e m sono coprimi se e solo se restituisce 1